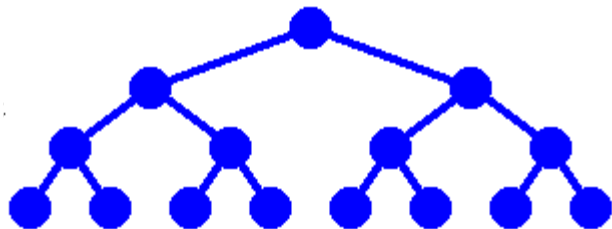


Trees



Trees

- *Trees* are used to impose a hierarchical structure on a collection of data items.
- A tree is a set of one or more nodes T such that:
 - there is a specially designated node called a root
 - The remaining nodes are partitioned into n *disjointed* set of nodes $T_1, T_2, \dots, T_n$, each of which is a tree.
- A **vertex or node** is a simple object that has a name and carry associated information.
- Sometimes, it is convenient to include among trees the *null tree*, a "tree" with no nodes
- An **edge** is a connection between two vertices.

Basics

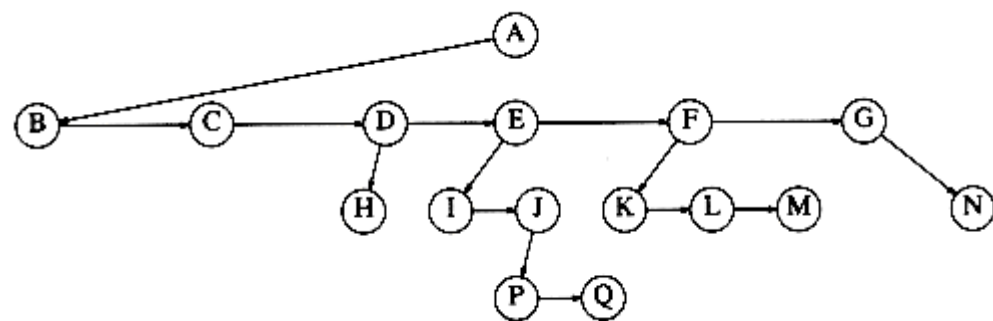
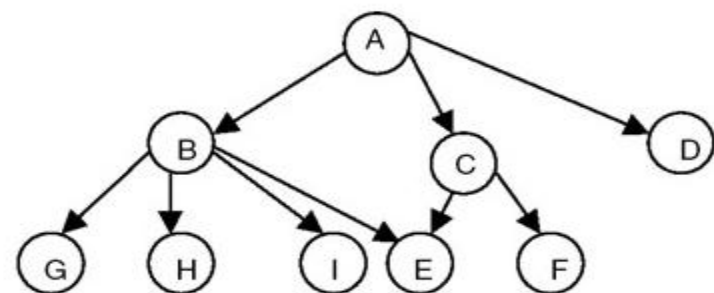
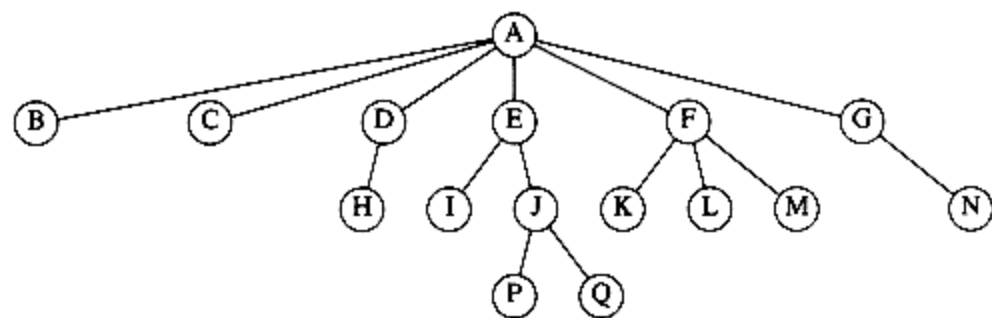
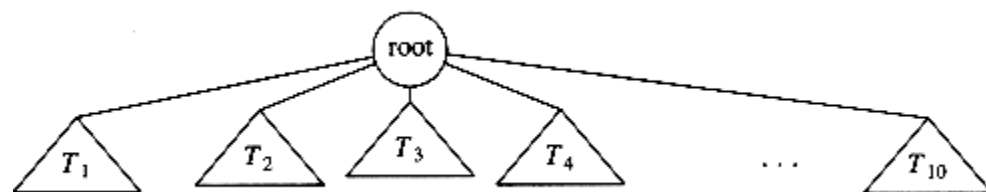
- Why Tree?
 - Linear access time $O(n)$ for linked list
 - Need a better structure
 - Tree is alternate for linear data arrangement with $O(\log n)$
- Binary Search Tree is popular structure for data representation

Definitions

- A **tree** is a collection of nodes. The collection can be empty. Otherwise, a tree consists of a distinguished node r , called the **root**, and zero or more (sub)trees $T_1, T_2, \dots, T_k$, *each of whose roots are connected by a directed edge to r*
- *Alternate Definition:*

A tree is a set of one or more nodes T such that there is a specially designated node called a root

The remaining nodes are partitioned into n *disjointed* set of nodes $T_1, T_2, \dots, T_n$, each of which is a tree.



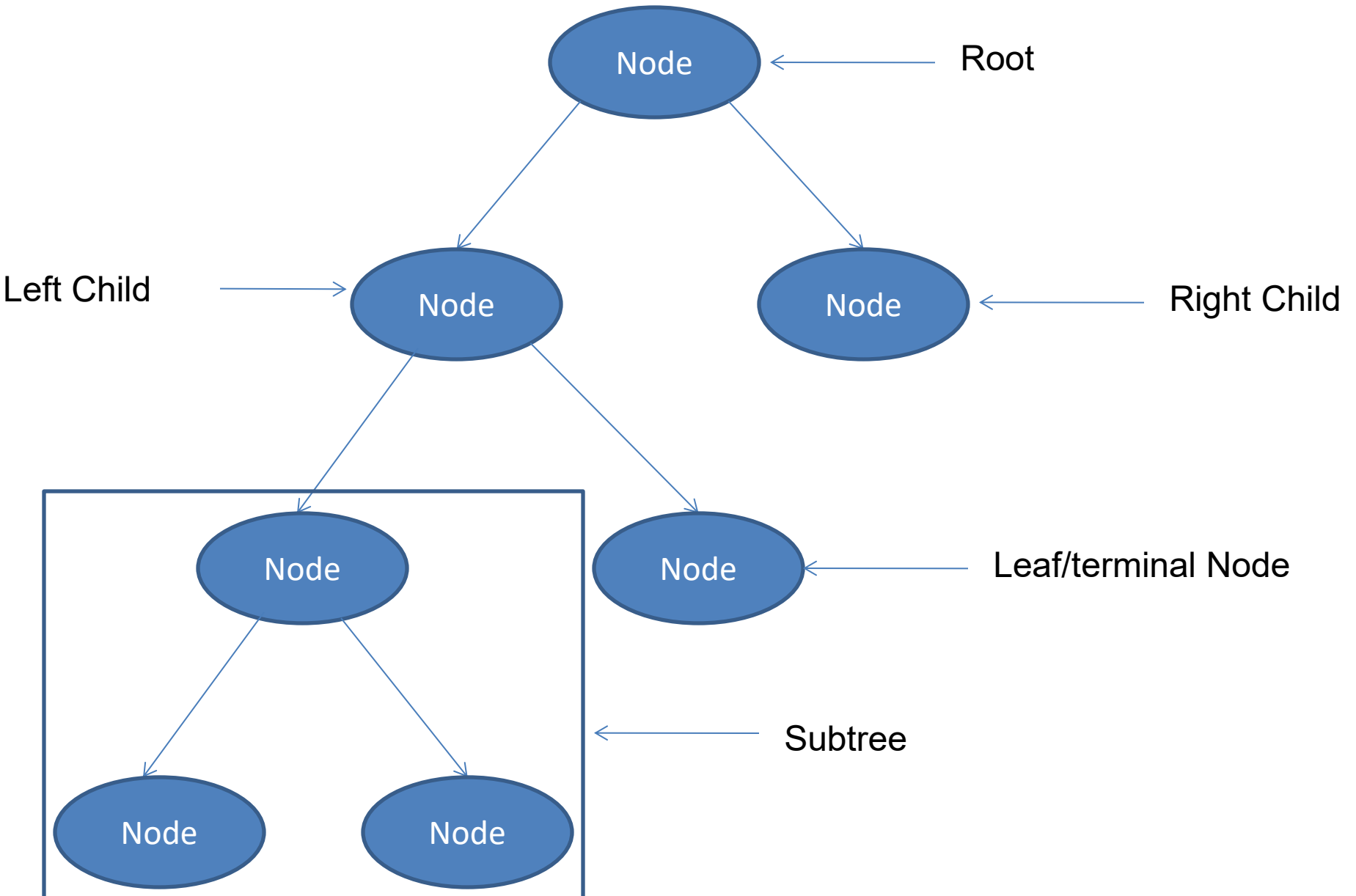
Trees

- A **node** may not have children, such node is known as **Leaf (terminal node)**.
- **Degree** = number of branches originating from the given node
- The ***height** of a node in a tree is the length of a longest path from the node to a leaf.*
- The ***height of a tree** is the height of the root.*
- *The **depth** of a node is the length of the unique path from the root to that node.*

Definitions -- 2

- Root of subtree is **child** of r and r is **parent** of each subtree's root
- Tree with N node contains $N-1$ edges
- Node with no children – **leaf**
- Nodes at same parent- **siblings**
- **Path (from n_1 to n_k)** -- sequence of nodes from n_1 to n_k
- **Length** – number of edges on a path
- **Depth** (of node n_i) – length of unique path from root to node n_i
- **Height** (of node n_i) - length of longest path from n_i to a leaf
- Root (depth 0) Leaf (height 0)
- **Level** – root 1 , subsequent descendants increment level by 1 for each descendent

Trees



Tree Implementation

```
typedef struct tree_node *tree_ptr;  
struct tree_node  
{  
    element_type element;  
    tree_ptr first_child;  
    tree_ptr next_sibling;  
};
```

Binary Tree

Definition - Tree in which each node can at max two children

Full Binary Tree - A full binary tree is a binary tree of level k having $2^k - 1$ nodes

Complete Binary Tree - A complete binary tree of level k is a tree with n nodes in which these n nodes can be numbered sequentially from 1 to n , as if it would have been the first n nodes in a full binary tree of depth k

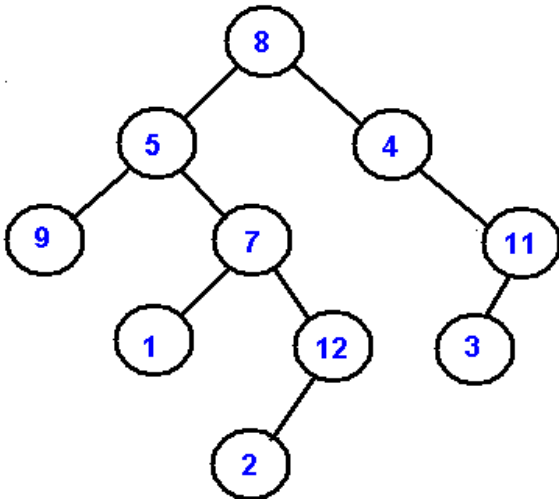
Full and Complete Binary tree can be represented using Array

Linked List Implementation:

```
typedef struct tree_node *tree_ptr;  
struct tree_node  
{  
    element_type element;  
    tree_ptr left;  
    tree_ptr right;  
};  
typedef tree_ptr TREE;
```

Tree Traversal

- In Order – 9 5 1 7 2 12 8 4 3 11
- Pre Order – 8 5 9 7 1 12 2 4 11 3
- Post Order – 9 1 2 12 7 5 3 11 4 8
- Level order – 8 5 4 9 7 11 1 12 3 2



Inorder

```
void Inorder( TREE T )  
{  
    if( T != NULL )  
    {  
        Inorder( T->left );  
        print_element( T->element );  
        Inorder( T->right );  
    }  
}
```

Note: Similar code for preOrder and PostOrder

Level order Traversal

Void levelorder(TREE T)

```
{
    Q = empty queue;
    if(T==NULL)
        return;
    enqueue(Q,T)
    while(Q is not empty)
    {
        node = dequeue(Q);
        print_element (node->element);
        if(node->lptr != NULL) ***
            enqueue(Q,node->left);
        if(node->rptr != NULL)
            enqueue(Q,node->right);
    }
}
```

*** - Tree is considered as binary tree. For generic tree in place of if statement we can use for loop to push all the children on current node.

Binary Search Trees

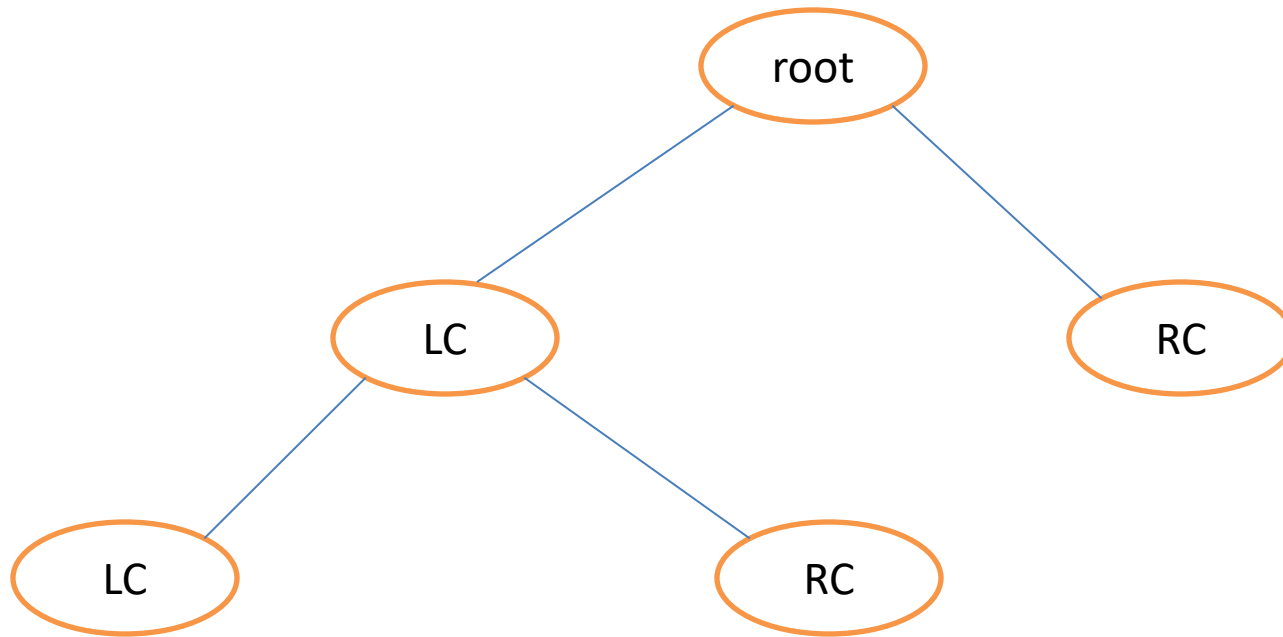
Binary Search Tree

- A *binary search tree* is a binary tree that may be empty, and every node must contain an identifier. An identifier of any node in the left subtree is less than the identifier of the root. An identifier of any node in the right subtree is greater than the identifier of the root. Both the left subtree and right subtree are binary search trees.
- The binary search tree is basically a binary tree, and therefore it can be traversed in inorder, preorder, and postorder
- If we traverse a binary search tree in inorder and print the identifiers contained in the nodes of the tree, we get a sorted list of identifiers in ascending order.
- A binary search tree is an important search structure.

Properties

- Elements or keys stored should be unique - No duplication is allowed
- Value of root element should always be greater than the left child
- Root should always be less than right child
- Subtree should also satisfy the above conditions

BST



Root element $>$ left child
Root element $<$ right child

If a tree satisfies all the above 4 properties, then it is said to be a binary search tree.

BST implementation

Declaration

```
typedef struct tree_node *tree_ptr;  
struct tree_node  
{  
    element_type element;  
    tree_ptr left;  
    tree_ptr right;  
};  
typedef tree_ptr SEARCH_TREE;
```

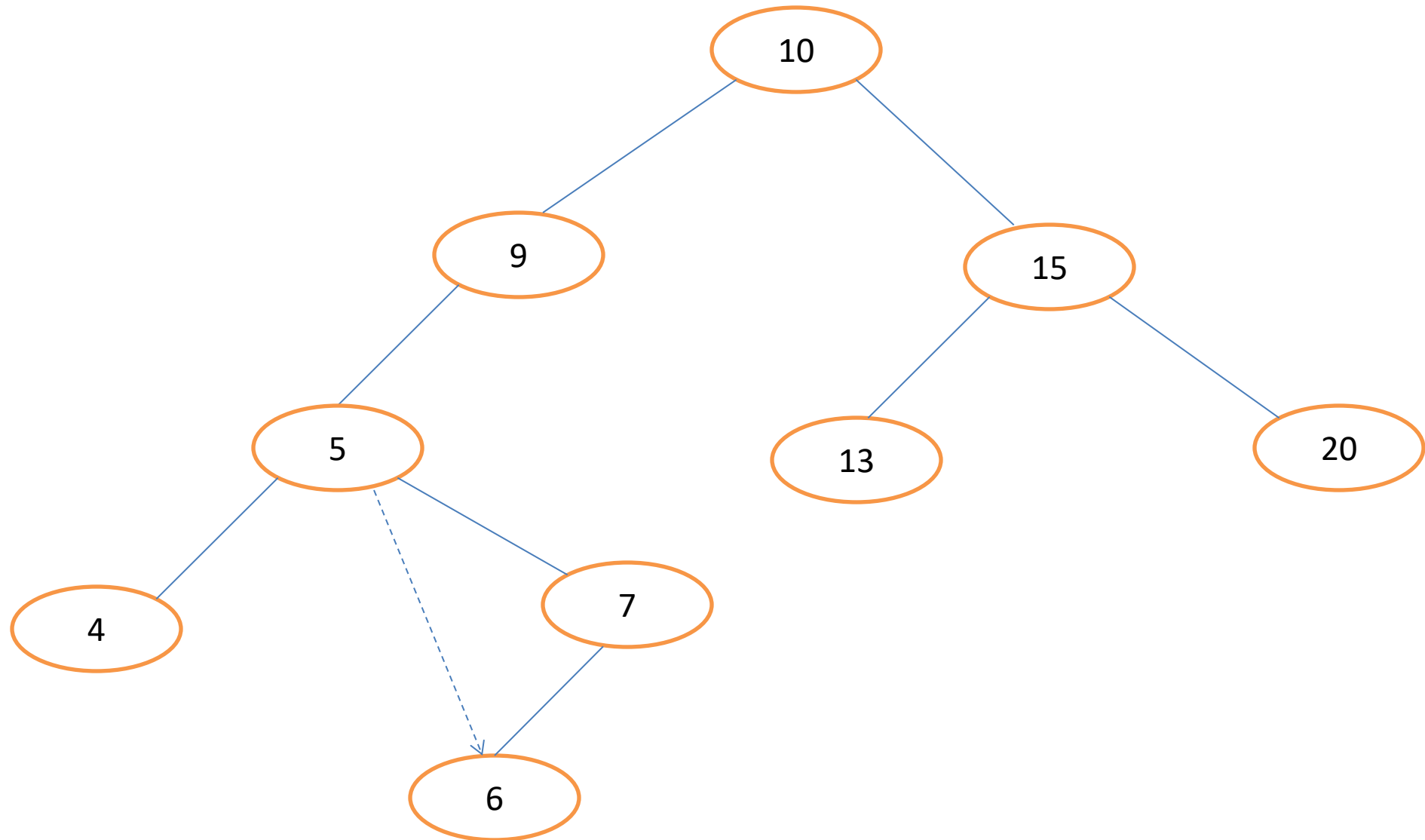
Insert an element in BST

[hint : we use inorder – ie. Left, root, right]

Struct btree*create bst(struct btree*root, int ele)

```
{
    if (root ==null)
    {
        root=(struct btree*)malloc(sizeof(struct btree))
        root->info=ele;
        root->lchild =root->rchild=null
    }
    else if(root->info==ele)
        printf("node already present");
    else if (root->info >ele)
        root->lchild=create bst(root->lchild,ele)
    else if(root->info<ele)
        root->rchild=create bst(root->rchild, ele)
    return root;
}
```

Delete an element from a tree such that after deletion, the tree must be in a BST



Case 1: If the node to be deleted is a leaf node.

Nodes with values -- 5,7,13,20

Case 2: Delete the root node with value 10: Insert an element in that node and the value should be chosen from the right node. The chosen value must be the minimum value on the right subtree node with value 13.

Case 3: The node we want to delete has only one subtree; either to the left or to the right.

```

tree_ptr delete( element_type x, SEARCH_TREE T )
{
    tree_ptr tmp_cell, child;
    if( T == NULL )
        error("Element not found");
    else
        if( x < T->element ) /* Go left */
            T->left = delete( x, T->left );
        else
            if( x > T->element ) /* Go right */
                T->right = delete( x, T->right );
            else /* Found element to be deleted */
                if( T->left && T->right ) /* Two children */
                { /* Replace with smallest in right subtree */
                    tmp_cell = find_min( T->right );
                    T->element = tmp_cell->element;
                    T->right = delete( T->element, T->right );
                }
            else /* One child */
            [
                tmp_cell = T;
                if( T->left == NULL ) /* Only a right child */
                    child = T->right;
                if( T->right == NULL ) /* Only a left child */
                    child = T->left;
                free( tmp_cell );
                return child;
            ]
    return T;
}

```

Find an element in BST

```
tree_ptr find( element_type x, SEARCH_TREE T )
{
    if( T == NULL )
        return NULL;
    if( x < T->element )
        return( find( x, T->left ) );
    else
        if( x > T->element )
            return( find( x, T->right ) );
        else
            return T;
}
```

FindMin and FindMax

```
tree_ptr find_min( SEARCH_TREE T )
{
    if( T == NULL )
        return NULL;
    else
        if( T->left == NULL )
            return( T );
        else
            return( find_min ( T->left ) );
}
```

```
tree_ptr find_max( SEARCH_TREE T )
{
    if( T != NULL )
        while( T->right != NULL )
            T = T->right;
    return T;
}
```

Insert to BST

```
tree_ptr insert( element_type x, SEARCH_TREE T )
{
    if( T == NULL )
    { /* Create and return a one-node tree */
        T = (SEARCH_TREE) malloc ( sizeof (struct tree_node) );
        if( T == NULL )
            fatal_error("Out of space!!!");
        else
        {
            T->element = x;
            T->left = T->right = NULL;
        }
    }
    else
    if( x < T->element )
        T->left = insert( x, T->left );
    else
    if( x > T->element )
        T->right = insert( x, T->right );
    /* else x is in the tree already. We'll do nothing */
    return T;
}
```

Delete a node from tree

```
struct btree* delete_node(struct btree* root, int ele)
{
    struct btree* temp;
    if (root == null)
        Printf("element not found\n");
    else if (ele < root->info) /*go left*/
        root->lchild = delete_node(root->lchild, ele);
    else if (ele > root->info) /*go right*/
        root->rchild = delete_node(root->rchild, ele);
    else if (root->lchild && root->rchild) /*two children*/
    {
        /*replace with the smallest value in the right subtree*/
        temp = find_min(root->rchild);
        root->info = temp->info;
        root->rchild = delete_node(root->rchild, root->info);
    }
}
```

Delete a node from tree (contd.)

```
else
{
    temp=root;
    if(root->lchild==null)
        root=root->rchild;
    else if(root->rchild==null)
        root=root->lchild;

    free(temp);
}
return root;
}
```